

CN Theory Assignment01 Submission

Name: Yash Raj

Roll Number: 24k-0737

Section A:

Code:

```
import socket
import struct
import random
import time

DNS_PORT = 53
BUFFER_SIZE = 4096
TIMEOUT = 5

# Query construction
def encode_domain_name(domain):
    domain = domain.rstrip(".")
    encoded = b""

    for label in domain.split("."):
        if not label:
            raise ValueError("Invalid domain name (empty label).")

        label_bytes = label.encode("ascii")
        if len(label_bytes) > 63:
            raise ValueError("Domain label is too long (max 63 bytes).")

        encoded += struct.pack("B", len(label_bytes))
        encoded += label_bytes

    if len(encoded) > 253:
        raise ValueError("Domain name is too long (max 253 bytes).")

    encoded += b"\x00" # root terminator
    return encoded
```

```

def sanitize_domain(raw_domain):
    domain = raw_domain.strip()

    for scheme in ("https://", "http://"):
        if domain.lower().startswith(scheme):
            domain = domain[len(scheme):]

    domain = domain.split("/")[0] # drop any path
    return domain.strip()

def build_dns_query(domain):
    transaction_id = random.randint(0, 65535)
    flags = 0x0100 # QR=0 (query), Opcode=0, RD=1 (recursion desired)

    header = struct.pack(
        "!HHHHHH",
        transaction_id,
        flags,
        1,
        0,
        0,
        0
    )

    qname = encode_domain_name(domain)
    question = qname + struct.pack("!HH", 1, 1) # QTYPE=A, QCLASS=IN

    return header + question, transaction_id

# Response parsing
def read_domain_name(data, offset):
    """
    Decode a domain name starting at `offset`, following DNS message
    compression pointers (0xC0 prefix) where present.
    Returns (name, offset_after_name_in_original_stream).
    """

```

```

labels = []
original_offset = offset
jumped = False

while True:
    if offset >= len(data):
        raise ValueError("Malformed DNS response (name out of
bounds).")

    length = data[offset]

    if length == 0:
        offset += 1
        break

    if (length & 0xC0) == 0xC0: # compression pointer
        if offset + 1 >= len(data):
            raise ValueError("Malformed DNS pointer.")

        pointer = ((length & 0x3F) << 8) | data[offset + 1]

        if not jumped:
            original_offset = offset + 2

        offset = pointer
        jumped = True
        continue

    if length > 63:
        raise ValueError("Invalid DNS label length.")

    offset += 1

    if offset + length > len(data):
        raise ValueError("Malformed DNS response (label out of
bounds).")

    label = data[offset:offset + length].decode("ascii",
errors="replace")
    labels.append(label)

```

```

        offset += length

    name = ".".join(labels)
    return (name, original_offset) if jumped else (name, offset)

def parse_dns_response(data, expected_transaction_id):

    if len(data) < 12:
        raise ValueError("DNS response is too short.")
    (
        transaction_id,
        flags,
        question_count,
        answer_count,
        authority_count,
        additional_count
    ) = struct.unpack("!HHHHHH", data[:12])

    if transaction_id != expected_transaction_id:
        raise ValueError("Transaction ID mismatch (ignoring stray
packet).")

    qr = (flags >> 15) & 1
    authoritative = (flags >> 10) & 1
    recursion_desired = (flags >> 8) & 1
    recursion_available = (flags >> 7) & 1
    rcode = flags & 0xF

    if qr != 1:
        raise ValueError("Received a query, not a response.")

    status_codes = {
        0: "NOERROR",
        1: "FORMERR",
        2: "SERVFAIL",
        3: "NXDOMAIN",
        4: "NOTIMP",
        5: "REFUSED"
    }

```

```

status = status_codes.get(rcode, f"UNKNOWN ({rcode})")

print(f"Transaction ID: {transaction_id}")
print(f"Response Status: {status}")
print(f"Authoritative: {bool(authoritative)}")
print(f"Recursion Desired: {bool(recursion_desired)}")
print(f"Recursion Available: {bool(recursion_available)}")

if rcode != 0:
    return

#Question section
offset = 12
if question_count > 0:
    query_name, offset = read_domain_name(data, offset)

    if offset + 4 > len(data):
        raise ValueError("Malformed question section.")

    query_type, query_class = struct.unpack("!HH", data[offset:offset
+ 4])

    offset += 4

    print(f"Query Name: {query_name}")
    print(f"Query Type: {query_type} (A)")
    print(f"Query Class: {query_class} (IN)")

# Answer section
print(f"Answer Records: {answer_count}")
if answer_count == 0:
    print("No answer records found.")
    return

found_a_record = False

for i in range(answer_count):
    answer_name, offset = read_domain_name(data, offset)

    if offset + 10 > len(data):
        raise ValueError("Malformed answer record.")

```

```

        record_type, record_class, ttl, data_length = struct.unpack(
            "!HHIH", data[offset:offset + 10]
        )
        offset += 10

    if offset + data_length > len(data):
        raise ValueError("Malformed record data.")

    record_data = data[offset:offset + data_length]
    offset += data_length

    print(f"\nAnswer {i + 1}")
    print(f"Name: {answer_name}")
    print(f"Type: {record_type}")
    print(f"Class: {record_class}")
    print(f"TTL: {ttl} seconds")

    if record_type == 1 and data_length == 4: # A record
        ip_address = socket.inet_ntoa(record_data)
        print(f"IPv4 Address: {ip_address}")
        found_a_record = True
    elif record_type == 5: # CNAME, informational only
        cname, _ = read_domain_name(data, offset - data_length)
        print(f"CNAME points to: {cname}")

    if not found_a_record:
        print("\nNo IPv4 A record found in the answer section.")

# Networking
def dns_lookup(domain, dns_server):
    domain = sanitize_domain(domain)
    try:
        query, transaction_id = build_dns_query(domain)
    except (ValueError, UnicodeError) as error:
        print(f"Invalid domain name: {error}")
    return

```

```

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.settimeout(TIMEOUT)

try:
    print(f"\nDomain: {domain}")
    print(f"DNS Server: {dns_server}")
    print(f"Port: {DNS_PORT}")
    print("Protocol: UDP")

    sock.sendto(query, (dns_server, DNS_PORT))

    deadline = time.time() + TIMEOUT
    while True:
        remaining = deadline - time.time()
        if remaining <= 0:
            raise socket.timeout()

        sock.settimeout(remaining)
        response, addr = sock.recvfrom(BUFFER_SIZE)

        # Ignore replies that didn't come from the server we queried.
        if addr[0] != dns_server:
            continue

        try:
            parse_dns_response(response, transaction_id)
            break
        except ValueError as error:
            if "Transaction ID mismatch" in str(error):
                continue
            raise

except socket.timeout:
    print("DNS request timed out.")
except ValueError as error:
    print(f"Error: {error}")
except OSError as error:
    print(f"Socket error: {error}")
finally:
    sock.close()

```

```
# Main program
def main():
    dns_server = input("Enter DNS Server IP: ").strip()
    try:
        socket.inet_aton(dns_server)
    except socket.error:
        print("Invalid DNS server IP.")
        return
    while True:
        try:
            domain = input("\nEnter domain name or 'exit' to quit:
").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nExiting.")
            break
        if domain.lower() == "exit":
            break
        if not domain:
            print("Domain name cannot be empty.")
            continue
        dns_lookup(domain, dns_server)

if __name__ == "__main__":
    main()
```

Testing & Outputs:

i) Domain name: www.google.com

```
PS C:\Users\yashm\Desktop\CN-Assignment01_Yashraj(24k-0737)>
k-0737)\tempCodeRunnerFile.py"
Enter DNS Server IP: 8.8.8.8

Enter domain name or 'exit' to quit: www.google.com

Domain: www.google.com
DNS Server: 8.8.8.8
Port: 53
Protocol: UDP
Transaction ID: 65201
Response Status: NOERROR
Authoritative: False
Recursion Desired: True
Recursion Available: True
Query Name: www.google.com
Query Type: 1 (A)
Query Class: 1 (IN)
Answer Records: 8
```

```
Answer 1
Name: www.google.com
Type: 1
Class: 1
TTL: 33 seconds
IPv4 Address: 142.251.155.119
```

```
Answer 2
Name: www.google.com
Type: 1
Class: 1
TTL: 33 seconds
IPv4 Address: 142.251.150.119
```

```
Answer 3
Name: www.google.com
Type: 1
Class: 1
TTL: 33 seconds
IPv4 Address: 142.251.157.119
```

```
Answer 4
Name: www.google.com
Type: 1
Class: 1
TTL: 33 seconds
IPv4 Address: 142.251.156.119
```

```
Answer 5
Name: www.google.com
Type: 1
Class: 1
TTL: 33 seconds
IPv4 Address: 142.251.153.119
```

```
Answer 6
Name: www.google.com
Type: 1
Class: 1
TTL: 33 seconds
IPv4 Address: 142.251.151.119
```

```
Answer 7
Name: www.google.com
Type: 1
Class: 1
TTL: 33 seconds
IPv4 Address: 142.251.154.119
```

```
Answer 8
Name: www.google.com
Type: 1
Class: 1
TTL: 33 seconds
IPv4 Address: 142.251.152.119
```

ii) Domain name: www.mit.edu

```
Enter DNS Server IP: 8.8.8.8

Enter domain name or 'exit' to quit: www.mit.edu

Domain: www.mit.edu
DNS Server: 8.8.8.8
Port: 53
Protocol: UDP
Transaction ID: 41090
Response Status: NOERROR
Authoritative: False
Recursion Desired: True
Recursion Available: True
Query Name: www.mit.edu
Query Type: 1 (A)
Query Class: 1 (IN)
Answer Records: 3
```

```
Answer 1
Name: www.mit.edu
Type: 5
Class: 1
TTL: 1097 seconds
CNAME points to: www.mit.edu.edgekey.net

Answer 2
Name: www.mit.edu.edgekey.net
Type: 5
Class: 1
TTL: 60 seconds
CNAME points to: e9566.dscb.akamaiedge.net

Answer 3
Name: e9566.dscb.akamaiedge.net
Type: 1
Class: 1
TTL: 20 seconds
IPv4 Address: 184.29.225.157
```

iii) Domain name: www.who.org

```
Enter domain name or 'exit' to quit: www.who.org
```

```
Domain: www.who.org
```

```
DNS Server: 8.8.8.8
```

```
Port: 53
```

```
Protocol: UDP
```

```
Transaction ID: 17635
```

```
Response Status: NOERROR
```

```
Authoritative: False
```

```
Recursion Desired: True
```

```
Recursion Available: True
```

```
Query Name: www.who.org
```

```
Query Type: 1 (A)
```

```
Query Class: 1 (IN)
```

```
Answer Records: 1
```

```
Answer 1
```

```
Name: www.who.org
```

```
Type: 1
```

```
Class: 1
```

```
TTL: 21600 seconds
```

```
IPv4 Address: 192.133.11.1
```

iv) Domain name: www.pakistan.gov.pk

```
Enter domain name or 'exit' to quit: www.pakistan.gov.pk
```

```
Domain: www.pakistan.gov.pk
```

```
DNS Server: 8.8.8.8
```

```
Port: 53
```

```
Protocol: UDP
```

```
Transaction ID: 40768
```

```
Response Status: NOERROR
```

```
Authoritative: False
```

```
Recursion Desired: True
```

```
Recursion Available: True
```

```
Query Name: www.pakistan.gov.pk
```

```
Query Type: 1 (A)
```

```
Query Class: 1 (IN)
```

```
Answer Records: 1
```

```
Answer 1
```

```
Name: www.pakistan.gov.pk
```

```
Type: 1
```

```
Class: 1
```

```
TTL: 600 seconds
```

```
IPv4 Address: 203.101.184.84
```